Rust Safety Invariants for RTIC with Readers-Writer Locks

Valhe Kouneli
Unit of Computing Sciences
Tampere University
Tampere, Finland
valhe.kouneli@tuni.fi

Pawel Dzialo
Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden
pawel.dzialo@ltu.se

Per Lindgren
Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden
per.lindgren@ltu.se

***Abstract*—The RTIC framework enables safe concurrent programming in Rust. By leveraging the Rust type system, programs that violate memory safety are rejected by the compiler. In this paper, we review the resource-proxy design of the Rust-based RTIC framework and highlight type-system features that enable compile-time enforcement of memory safety. Moreover, we introduce an API extension that supports readers-writer locks and show that the proposed API enforces Rust's memory-safety invariants at compile time.**

***Index Terms*—Real-time systems, Safety-critical systems, Memory safety, Rust programming language, RTIC framework**

I. Introduction

The safety and security of systems rely increasingly on the behavior of their software stacks. Modern compiler back-ends optimize the code under the assumption that programs are well-formed. In fact, in case of programs having undefined behavior (UB), the compiler is free to generate **any** code along the path leading to up to the point of UB without any regard to the original program semantics[1]. Even worse, the compiler has no obligation to inform the programmer about the presence of UB, thus a program may pass compilation without any warnings or errors and yet express arbitrary behavior at runtime[2]. In effect, for such programs all claims to safety and security are void! This is unfortunately the case even for ARM Trust-Zone based systems (as well as PMP equipped RISC-V systems), where the hardware protection mechanisms are not going to help you if the code inside of the kernel/trusted zone has UB.

In this paper we focus on a class of UB caused by memory safety violations. To this end, Section II reviews the Rust language and its guarantees to memory safety. In Section III we review the RTIC framework and how the Rust type system is leveraged to ensure memory safety at compile time. In Section IV, we introduce an RTIC API extension that supports readers-writer locks for read-write resources, a special case of multi-unit resources, allowing concurrent readers and thereby reducing unnecessary blocking. We show that the proposed API continues to enforce Rust's memory-safety invariants at compile time. Finally, we conclude the work and contributions in Section VI.

II. Rust memory safety

The Rust programming language[3] enforces strong memory safety guarantees, unless the programmer explicitly opts out by marking code as `unsafe`[4].

For a majority of program constructs the Rust compiler can verify memory safety at compile time, and reject programs that violate the memory safety rules. In case safety cannot be statically verified, the compiler will inject runtime checks that halt execution (**panic**), *before* the program runs into UB. In this way, Rust ensures that code always runs with defined behavior. This is in stark contrast to C/C++ where it is completely up to the programmer to ensure defined behavior, and thus positions Rust in a unique and advantageous position for safety and security-critical systems.

A. Rust Memory Safety Invariants

The Rust type system is based on an **ownership** model, the principle of which is ensuring that each piece of data has a single owner at any given time. The borrowing mechanism allows for temporary access to data without transferring ownership, under the following rules:

- there can be either one mutable reference, or any number of immutable references to a piece of data at a time, and
- references must always be valid.

These rules are enforced at compile time by the Rust compiler. In this work, we focus on the first rule (as data initialization is out of scope), and refer to it as the *Rust memory safety invariants*.

B. Bare metal systems

In context of bare metal systems, we typically need to

- access the underlying hardware (raw memory accesses),
- share mutable data between concurrent tasks (e.g., interrupt handlers).

As they are outside the control of the Rust compiler, raw memory accesses and sharing mutable data are inherently `unsafe`.

Rust provides a mechanism for marking code blocks as `unsafe`, allowing the programmer to explicitly opt out of the Rust safety guarantees, and thus access the underlying hardware or share mutable data between concurrent tasks. The soundness of `unsafe` code relies on the programmer upholding the invariants from section Section II.

Notice that, compared to traditional C/C++, we are still in a vastly better position: only explicitly marked **unsafe** code blocks require manual review and verification, whereas in C/C++ the entire code base is potentially unsafe. A key aspect of UB is its temporal nature: the effects of UB may be not be directly observable, but may manifest at some later point in time. As UB is permissible at C/C++ language level, the best one can hope for is that safety violations lead to immediate and observable effects (like halting the system). Unless proven correct through formal verification, C/C++ code bases are inherently unsafe.

III. RTIC framework

The RTIC framework[5] is designed to provide concurrent access to shared mutable data without the need of any **unsafe** code. By leveraging on the Rust type system, memory safety is guaranteed at compile time, leaving the programmer to focus on the application logic. Access to underlying hardware can be done through (internally **unsafe**) pre-validated abstractions.

RTIC is a Domain Specific Language (DSL) extending Rust with a Stack Resource Policy (SRP) based concurrency model for bare metal programming. RTIC has since its release (2017, *cortex_m_rtic*) gained popularity (with ~1 million downloads accumulatively) and is now widely used in production systems (e.g., at Volvo Cars[6], Nitrokey[7] (Trussed), and considered at the European Space Agency[8]).

Leveraging Rust procedural macros, the RTIC framework:

- parses the application into an Abstract Syntax Tree (AST) model,
- performs static analysis of the model (SRP-based resource ceiling analysis etc.),
- generates code that is compiled to a stand-alone binary.

Run-time overhead is in Rust terms *zero-cost*[1], where the generated binary efficiently exploits the underlying hardware for scheduling and resource protection without any non-necessary overhead.[2] In fact, one can even claim RTIC to be *sub-zero-cost* as outperforming hand-written implementations of the same application logic. This is possible as the static analysis allows for optimizations of the entire application model, which is typically out of reach for a human programmer.

The key to RTIC's guaranteed memory safety is its underlying resource-proxy design, in which shared resources are represented by proxies that enforce Rust's ownership and borrowing rules.

In the following, we will review key design aspects that ensure the memory-safety invariants of Rust. For the sake of brevity, details concerning SRP compliance are deliberately omitted.

[1]Rust *zero-cost*, implies that no unnecessary runtime overhead is introduced, not the the cost is an absolute zero[9].

[2]In addition to *zero-cost* interrupt bound hardware tasks, RTIC v1 supports optional software tasks. The latter rely an external library for concurrent queues (the *heapless* crate[10]), which while being highly efficient do not claim to be *zero-cost* to the general problem of concurrent queues.

A. RTIC-core, Mutex trait

The RTIC-core library defines the `Mutex` abstraction, a Rust trait that defines the user facing API for accessing shared resources.

```rust
pub trait Mutex {
    /// Data protected by the mutex
    type T;

    /// Creates a critical section and grants temporary access to the protected data
    fn lock<R>(&mut self, f: impl FnOnce(&mut Self::T) -> R) -> R;
}
```

The `lock` method takes a closure that receives a mutable reference to the protected data and returns a result.

B. Mutex trait implementation

For each shared resource in the system, the RTIC framework generates a concrete implementation of the `Mutex` trait, which ensures safe concurrent access (i.e, exclusive access) that is based on the priority of the tasks accessing it.

To illustrate the principle, we side-step the RTIC framework, and implement the `Mutex` trait manually.

```rust
pub struct TestMutex<T> {
    data: T,
}

impl<T> Mutex for TestMutex<T> {
    type T = T;

    fn lock<R>(&mut self, f: impl FnOnce(&mut Self::T) -> R) -> R {
        // In a real implementation, this would be generated to ensure exclusive access to the data
        // As dictated by SRP, raise system ceiling to that of the highest priority task accessing the resource
        f(&mut self.data)
        // As dictated by SRP, restore (old) system ceiling
    }
}
```

A real generated counterpart would implement the necessary logic to ensure exclusive access to the underlying data, and would be uniquely generated based on the priority of the tasks accessing it.

C. Rust borrowing invariants

In the following we will illustrate how the `Mutex` implementation successfully leverages the Rust type system to reject Rust safety invariant violations at compile time.

a) *Example Valid Access:*

To illustrate the principle, we again side-step the RTIC framework and manually implement the `Mutex` trait for a simple data structure, `NonAtomicU32` (which cannot be safely shared between concurrent tasks without resource protection in place).

An example of valid resource use:

```rust
// this would be marked as a resource in RTIC
#[derive(Copy, Clone)]
struct NonAtomicU32 {
    x: u32,
    y: u32,
    // additional fields
}

...
// mutex proxy is passed through the task context, and is guaranteed to be unique for the task
let d = mutex.lock(|data| {
    data.x += 1; // data : &mut NonAtomicU32
    *data
});
// do something with the copied data
```

We can safely copy the underlying data in case the type implements the `Copy` trait[9], and return it from the closure.

b) *Example Leaking:*

Attempts to leak a reference to the underlying `NonAtomicU32` outside of the protected closure are rejected by the compiler.

```rust
let d = mutex.lock(|data| {
    data.x += 1;
    data // attempt to leak the reference
});
```

The compiler error (Appendix A, Listing 1) points out that the lifetime of `data` ends at the return point of the closure. If the reference was leaked, Rust's borrowing invariants would be violated, causing a potential race condition: a leaked reference would be accessible concurrently from multiple tasks of the system outside of the protection of the `Mutex` lock.

c) *Mutex nesting:*

The Rust compiler will successfully reject mutable aliasing of the underlying data.

```rust
let d = mutex.lock(|data| {
    let d = mutex.lock(|data_inner| {
        data.x += 1; // access to underlying data
        data_inner.x += 1; // access to underlying data
    *data
    });
});
```

Notice here that both `data` and `data_inner` are mutable references to the *same* underlying data, which violates Rust's borrowing invariants introduced in Section II.A. This is also pointed out by the resulting compiler error (Appendix A, Listing 2).

d) *Mutex safety guarantees:*

The above examples together show that the `Mutex` implementation successfully leverages the Rust type system to reject safety violations at compile time:

- leaking of references (see Section III.C.b),
- mutable aliasing of the underlying data (see Section III.C.c).

IV. RTIC-RW, Readers-Writer Locks

In recent work, RTIC-RW has been proposed as an extension to the RTIC framework, supporting readers-writer locks that protect read-write resources (a special case of multi unit resources). It has been shown that readers-writer locks can be implemented in a safe manner without incurring any additional run-time overhead, thus providing a strict improvement over RTIC's current single-unit resource design.

In this work, we contribute the API design of RTIC-RW and show that its implementation successfully enforces the Rust memory-safety invariants (Section II.A) at compile time.

A. Proposed MutexRW trait

```rust
pub trait MutexRW {
    /// Data protected by the mutex
    type T;

    /// Creates a critical section and grants temporary access to the protected data
    fn read_lock<R>(&self, f: impl FnOnce(&Self::T) -> R) -> R;

    fn write_lock<R>(&mut self, f: impl FnOnce(&mut Self::T) -> R) -> R;
}
```

As described in Section II, Rust's borrowing rules allow either multiple immutable borrows or a single mutable borrow of any given value. The `read_lock` method takes `&self`, an immutable reference to the proxy, thus allowing multiple concurrent readers. The `write_lock` method takes `&mut self`, a mutable reference to the proxy, thus allowing only a single writer.

In the following section, we will illustrate how the `MutexRW` implementation successfully leverages the Rust type system to reject violations of Rust's memory-safety invariants at compile time.

a) *Example Valid Nested Access:*

We reuse the `NonAtomicU32` data structure from Section III, and implement the `MutexRW` trait for it.

```rust
...
// mutex proxy is passed through the task context, and is guaranteed to be unique for the task
let (x, y) = mutex_rw.read_lock(|data| {
    let y = mutex_rw.read_lock(|data_inner| {
        data_inner.y
    });
    (data.x, y)
});
// do something with the copied data (x, y)
```

Even if the `data` and `data_inner` are both references to the *same* underlying data, the nested `read_lock` follows Rust's borrowing invariants, and thus the compiler will accept the program. The Rust compiler concludes that we can safely nest read locks, as the `read_lock` method borrows the proxy in an immutable manner.

We can safely copy the underlying data. The fields are of type `u32` (which implements the `Copy` trait), and return it from the closure.

b) *Example Valid Write Access:*

This trivially follows the validation of the `Mutex` lock implementation, and is thus omitted for brevity.

c) *Example Leaking:*

Leaking rejection is strictly analogous to the `Mutex` implementation, and is thus omitted for brevity.

d) *MutexRW Read-Write nesting:*

Considering the following, faulty example:

```Rust
// mutex proxy is passed through the task context, and is guaranteed to be unique for the task
mutex_rw.read_lock(|data| {
    mutex_rw.write_lock(|data_inner| {
        data_inner.x += data.x;
        data_inner.y += data.x;
    });
});
```

Notice here that both `data` and `data_inner` are references to the *same* underlying data, with `data_inner` being a mutable reference, thus violating Rust's borrowing invariants. This is successfully caught by the Rust compiler (Appendix A, Listing 3).

e) *MutexRW Write-Read nesting:*

Inverting the lock acquisition order from the previous example:

```Rust
// mutex proxy is passed through the task context, and is guaranteed to be unique for the task
mutex_rw.write_lock(|data| {
    data.x += 1;
    mutex_rw.read_lock(|data_inner| {
        // access to underlying data
    });
});
```

Here, again, `data` and `data_inner` are references to the same data, with `data` being mutable. This violation is successfully caught by the Rust compiler, analogous to previous example.

f) *MutexRW safety guarantees:*

The above examples together show that the `MutexRW` implementation successfully leverages the Rust type system to reject Rust safety invariant violations at compile time:

- leaking of references, analogous to the original `Mutex` implementation (see Section III.C.b)
- mutable aliasing of the underlying data, by either read-write or write-read nesting (see Section IV.A.d and Section IV.A.e)

B. Proposed MutexR trait

The current RTIC framework can be modified to allow for an additional keyword in the task definition indicating that the task only reads the resource, and thus should be given a read-only proxy `MutexR` rather than a read-write proxy `MutexRW`.

```Rust
// read-only proxy is passed through the task context, and is guaranteed to be unique for the task
pub trait MutexR {
    /// Data protected by the mutex
    type T;

    /// Creates a critical section and grants temporary access to the protected data
    fn read_lock<R>(&self, f: impl FnOnce(&Self::T) -> R) -> R;
}
```

a) *Proposed MutexR safety guarantees:*

A read-only proxy can only be used to read the underlying data. It will analogously to `MutexRW` allow `read_lock` nesting and reject leaking of references. Examples are omitted for brevity.

V. Discussion and Future Work

An observation is that, while such an operation is not safe in Rust, SRP itself permits a write lock to be acquired inside a read lock if no other job holds locks of the resource, while still preserving all scheduling guarantees provided by SRP.

Under SRP, RW resources are modeled as multi-unit resources, where the number of units corresponds to the number of jobs that may access the resource. A read operation requires acquiring one unit, and a write lock requires acquiring all units. To safely perform a write operation, it suffices for the task to have acquired all resource units and therefore mutually exclude all other tasks from accessing the resource.

SRP guarantees that a task will not start if it risks being unable to acquire all the resources it requires. Consequently, a task that both reads from and writes to a resource will only execute when no other task holds a lock on that resource. SRP could therefore allow a read lock to be promoted to a write lock and demoted back to a read lock once the write operation is over. However, a safe Rust implementation of a write lock nested inside a read lock would need to invalidate the immutable reference obtained from the read lock somehow before creating a mutable reference required by the write lock.

Reference demotion is possible without any changes to the proposed MutexRW API as follows:

```Rust
let x = mutex_rw.write_lock(|data| {
    data.x += 1; // mutable reference to underlying data
    let data = &*data; // Rust re-borrow to obtain an immutable reference
    // data += 1; // This line is invalid because `data` is an immutable reference
    data.x
})
```

The Rust compiler would successfully reject the demoted reference from being used to modify the underlying data. However, the demotion will not be visible to the RTIC framework at run-time. Thus, the code after demotion will still be executed under the protection ceiling of the write lock, with the effect of potentially blocking other, higher priority readers.

Still, for reasons of code clarity/safety, demotion may be useful and would not break safety invariants.

The situation of promotion is more complex, as re-borrowing an immutable reference to obtain a mutable reference would directly violate the Rust borrowing invariants.

To this end, we might consider an API extension to allows promotion of a read lock to a write lock. This however is out of scope for this paper, and is left for future work.

Beyond the RW case, the underlying SRP theory allows for general multi-unit resources. However, implementing the general case would require run-time tracking of the number of units currently held, and would thus incur run-time overhead. Whether this is acceptable in practice is left for future work.

VI. Conclusions

In this paper we have reviewed the resource-proxy design of the Rust-based RTIC framework and highlighted the type system features that enable compile-time safety guarantees. We have also introduced an API extension that enables the use of readers-writer locks and shown that the proposed API successfully enforces Rust's memory safety invariants at compile time.

While RTIC-RW provides a strict improvement in scheduling properties over RTIC's current single unit resource design, prior work lacked an API design that ensures compile-time rejection of violations of Rust's safety invariants. In this work, we have detailed the API design of the underlying `MutexR/RW` and shown that its implementation successfully enforces these invariants at compile time.

Appendix A
Compiler Error Messages

Compiler error messages have been slightly reformatted for clarity, but are otherwise genuine compiler output. Notice that examples in the main text are excerpts from real code thus do not match error messages 1-1.

```
error: lifetime may not live long enough
  --> examples/mutex_leak.rs:18:9
15 |     let d = mutex.lock(|data| {
   |                         ----- return type of closure is &'2 mut NonAtomicU32
   |                         |
   |                         has type `&'1 mut NonAtomicU32`
...
18 |         data
   |         ^^^^ returning this value requires that `'1` must outlive `'2`
help: dereference the return value
18 |         *data
```

terminal

Listing 1. `Mutex` leaking error message

```
error[E0499]: cannot borrow `mutex` as mutable more than once at a time
  --> examples/mutex_nesting.rs:15:13
15 |       let d = mutex.lock(|data| {
   |               ^     ---- ------ first mutable borrow occurs here
   |               |     |
   |  _____________|     first borrow later used by call
   | |
16 | |         data.x += 1;
18 | |         let d = mutex.lock(|data| {
   | |                 ----- first borrow occurs due to use of `mutex` in closure
...  |
22 | |         *data
23 | |     });
   | |______^ second mutable borrow occurs here

error[E0499]: cannot borrow `mutex` as mutable more than once at a time
  --> examples/mutex_nesting.rs:15:24
15 |     let d = mutex.lock(|data| {
   |             ----- ---- ^^^^^^ second mutable borrow occurs here
   |             |     |
   |             |     first borrow later used by call
   |             first mutable borrow occurs here
...
18 |         let d = mutex.lock(|data| {
   |                 ----- second borrow occurs due to use of `mutex` in closure
```

terminal

Listing 2. `Mutex` nesting error message

```
error[E0502]: cannot borrow `mutex_rw` as mutable because it is also borrowed as immutable
  --> examples/mutex_rw_r_w.rs:15:24
15 |     mutex_rw.read_lock(|data| {
   |     -------- --------- ^^^^^^ mutable borrow occurs here
   |     |        |
   |     |        immutable borrow later used by call
   |     immutable borrow occurs here
17 |         mutex_rw.write_lock(|data_inner| {
   |         -------- second borrow occurs due to use of `mutex_rw` in closure
```

terminal

Listing 3. `MutexRW` read-write nesting error message

References

[1] L. Popescu and N. P. Lopes, "Exploiting Undefined Behavior in C/C++ Programs for Optimization: A Study on the Performance Impact," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, June 2025, doi: 10.1145/3729260.

[2] X. Wang, H. Chen, A. Cheung, A. M. Zuckerberg, and N. Zeldovich, "Undefined behavior: what happened to my code?," in *Proceedings of the 2012 Asia-Pacific Workshop on Systems*, in APSys '12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 1–6. doi: 10.1145/2349896.2349905.

[3] R. Team, "Rust Language." [Online]. Available: https://rust-lang.org/

[4] R. Team, "Rust - The Unsafe Keyword." [Online]. Available: https://doc.rust-lang.org/stable/reference/unsafe-keyword.html?highlight=unsafe#the-unsafe-keyword

[5] RTIC Team, "The RTIC Book (v1.0)." [Online]. Available: https://rtic.rs/1/book/en/

[6] Volvo, "Why Rust is actually good for your car." [Online]. Available: https://medium.com/volvo-cars-engineering/why-volvo-thinks-you-should-have-rust-in-your-car-4320bd639e09

[7] N. Team, "Nitro Key 3 - RTIC." [Online]. Available: https://github.com/Nitrokey/nitrokey-3-firmware/blob/main/components/boards/Cargo.toml

[8] R. Müller, P. Nehlich, and S. Klinkner, "Leveraging the Rust Programming Language for Space Applications," in *2024 IEEE Space Computing Conference (SCC)*, 2024, pp. 40–50. doi: 10.1109/SCC61854.2024.00011.

[9] S. Klabnik and C. Nichols, *The Rust Programming Language*. USA: No Starch Press, 2018.

[10] J. Aparicio Rivera, M. Lindner, and P. Lindgren, "Heapless : Dynamic Data Structures without Dynamic Heap Allocator for Rust," in *2018 IEEE 16TH INTERNATIONAL CONFERENCE ON INDUSTRIAL INFORMATICS (INDIN) :*, in IEEE International Conference on Industrial Informatics INDIN. 2018, pp. 87–94. doi: 10.1109/INDIN.2018.8472097.